Importing Libraries

In [42]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping
```

Loading Dataset

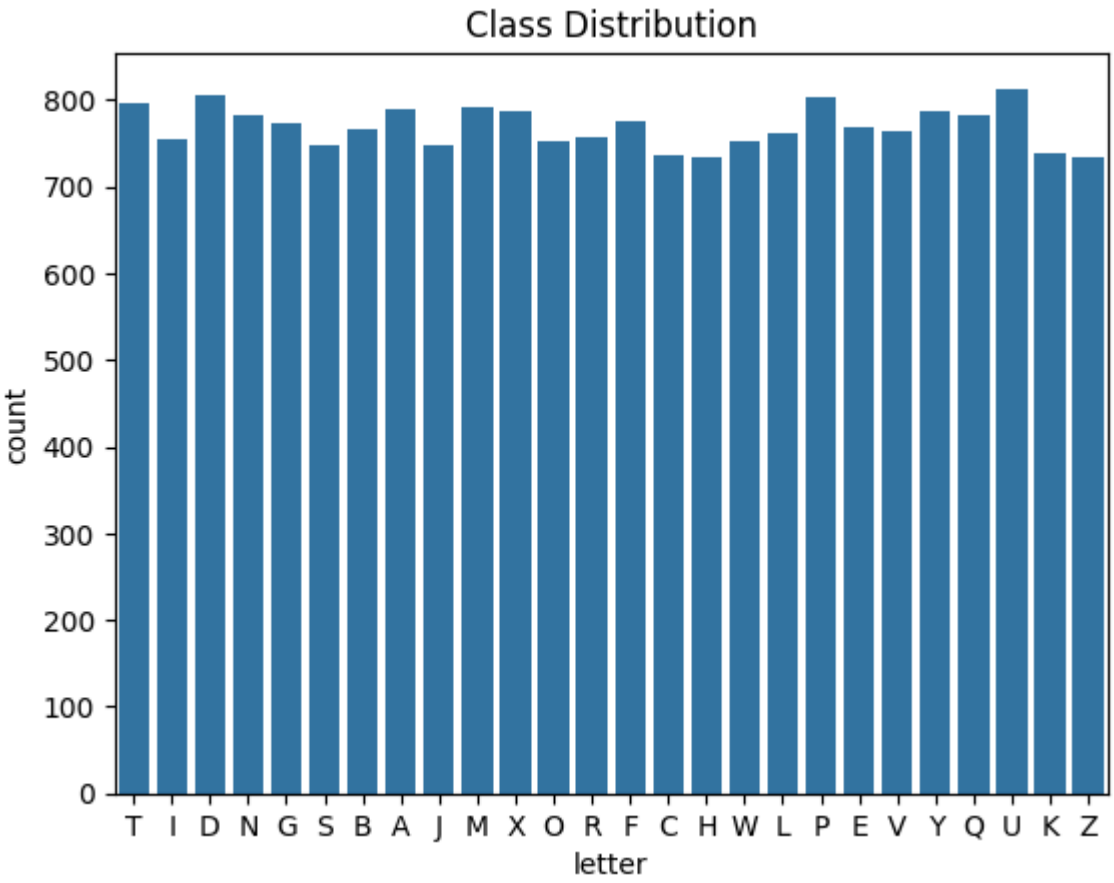
In [ ]:
```
cols = ["letter","x-box","y-box","width","height","onpix","x-bar","y-bar"
        "x2bar","y2bar","xybar","x2ybr","xy2br","x-ege","xegvy","y-ege","

df = pd.read_csv("../data/letter-recognition.data", names=cols)
```

In [12]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 17 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   letter  20000 non-null  str
 1   x-box   20000 non-null  int64
 2   y-box   20000 non-null  int64
 3   width   20000 non-null  int64
 4   height  20000 non-null  int64
 5   onpix   20000 non-null  int64
 6   x-bar   20000 non-null  int64
 7   y-bar   20000 non-null  int64
 8   x2bar   20000 non-null  int64
 9   y2bar   20000 non-null  int64
 10  xybar   20000 non-null  int64
 11  x2ybr   20000 non-null  int64
 12  xy2br   20000 non-null  int64
 13  x-ege   20000 non-null  int64
 14  xegvy   20000 non-null  int64
 15  y-ege   20000 non-null  int64
 16  yegvx   20000 non-null  int64
dtypes: int64(16), str(1)
memory usage: 2.6 MB
```

Exploratory Data Analysis

In [17]:
```
sns.countplot(x='letter', data=df)
plt.title('Class Distribution')
plt.show()
```



Data Preprocessing

In [18]:
```
X = df.drop('letter', axis=1)
y = df['letter']

# Label Encoding
le = LabelEncoder()
y = le.fit_transform(y)

# train and test data split.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# feature scaling.
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# one hot encoding.
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

Model Building

In [23]:
```
model = Sequential()

model.add(Dense(512, activation='relu', input_shape=(16,)))
model.add(Dropout(0.4))

model.add(Dense(256, activation='relu'))
model.add(Dropout(0.4))
```

```
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.4))

model.add(Dense(26, activation='softmax'))

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
/Users/parimal/VisualStudioCodeProjects/dl-assignments/venv/lib/python3.1
3/site-packages/keras/src/layers/core/dense.py:107: UserWarning: Do not pa
ss an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the mo
del instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model Training

In [24]:
```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop]
)
```

```
Epoch 1/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.4715 - loss: 1.7545
- val_accuracy: 0.7494 - val_loss: 0.8736
Epoch 2/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.6927 - loss: 0.9851
- val_accuracy: 0.8197 - val_loss: 0.6224
Epoch 3/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7527 - loss: 0.7873
- val_accuracy: 0.8562 - val_loss: 0.4967
Epoch 4/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.7867 - loss: 0.6667
- val_accuracy: 0.8719 - val_loss: 0.4163
Epoch 5/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8098 - loss: 0.5987
- val_accuracy: 0.8981 - val_loss: 0.3604
Epoch 6/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8258 - loss: 0.5469
- val_accuracy: 0.9053 - val_loss: 0.3212
Epoch 7/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8445 - loss: 0.4830
- val_accuracy: 0.9072 - val_loss: 0.2896
Epoch 8/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8571 - loss: 0.4577
- val_accuracy: 0.9197 - val_loss: 0.2642
Epoch 9/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8673 - loss: 0.4184
- val_accuracy: 0.9241 - val_loss: 0.2393
Epoch 10/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8719 - loss: 0.3969
- val_accuracy: 0.9259 - val_loss: 0.2347
Epoch 11/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8759 - loss: 0.3846
- val_accuracy: 0.9328 - val_loss: 0.2161
Epoch 12/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8854 - loss: 0.3603
- val_accuracy: 0.9381 - val_loss: 0.2054
Epoch 13/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8866 - loss: 0.3517
- val_accuracy: 0.9397 - val_loss: 0.1882
Epoch 14/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8940 - loss: 0.3288
- val_accuracy: 0.9450 - val_loss: 0.1842
Epoch 15/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8926 - loss: 0.3215
- val_accuracy: 0.9447 - val_loss: 0.1849
Epoch 16/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9002 - loss: 0.3131
- val_accuracy: 0.9481 - val_loss: 0.1735
Epoch 17/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9020 - loss: 0.3016
- val_accuracy: 0.9466 - val_loss: 0.1658
Epoch 18/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9091 - loss: 0.2785
- val_accuracy: 0.9459 - val_loss: 0.1596
Epoch 19/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9105 - loss: 0.2743
- val_accuracy: 0.9534 - val_loss: 0.1487
Epoch 20/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9136 - loss: 0.2684
- val_accuracy: 0.9538 - val_loss: 0.1456
```

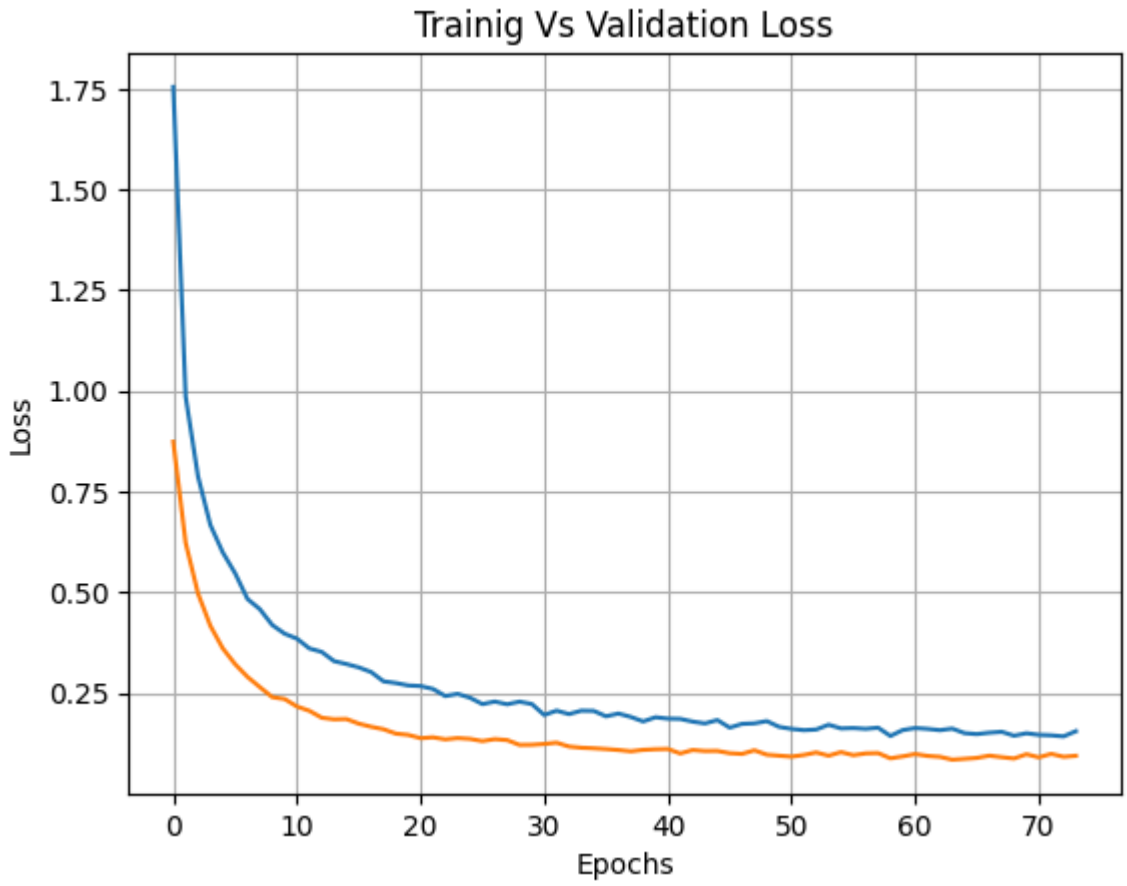
```
Epoch 21/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9130 - loss: 0.2668
- val_accuracy: 0.9566 - val_loss: 0.1374
Epoch 22/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9131 - loss: 0.2596
- val_accuracy: 0.9559 - val_loss: 0.1395
Epoch 23/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9208 - loss: 0.2420
- val_accuracy: 0.9550 - val_loss: 0.1345
Epoch 24/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9182 - loss: 0.2474
- val_accuracy: 0.9547 - val_loss: 0.1376
Epoch 25/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9238 - loss: 0.2378
- val_accuracy: 0.9563 - val_loss: 0.1357
Epoch 26/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9270 - loss: 0.2220
- val_accuracy: 0.9566 - val_loss: 0.1302
Epoch 27/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9281 - loss: 0.2284
- val_accuracy: 0.9550 - val_loss: 0.1350
Epoch 28/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9261 - loss: 0.2216
- val_accuracy: 0.9538 - val_loss: 0.1327
Epoch 29/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9248 - loss: 0.2280
- val_accuracy: 0.9588 - val_loss: 0.1207
Epoch 30/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9286 - loss: 0.2222
- val_accuracy: 0.9597 - val_loss: 0.1210
Epoch 31/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9347 - loss: 0.1950
- val_accuracy: 0.9581 - val_loss: 0.1231
Epoch 32/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9324 - loss: 0.2058
- val_accuracy: 0.9597 - val_loss: 0.1262
Epoch 33/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9334 - loss: 0.1972
- val_accuracy: 0.9597 - val_loss: 0.1167
Epoch 34/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9337 - loss: 0.2056
- val_accuracy: 0.9628 - val_loss: 0.1137
Epoch 35/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9337 - loss: 0.2051
- val_accuracy: 0.9603 - val_loss: 0.1123
Epoch 36/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9376 - loss: 0.1918
- val_accuracy: 0.9641 - val_loss: 0.1103
Epoch 37/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9366 - loss: 0.1986
- val_accuracy: 0.9647 - val_loss: 0.1081
Epoch 38/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9356 - loss: 0.1902
- val_accuracy: 0.9663 - val_loss: 0.1048
Epoch 39/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9407 - loss: 0.1790
- val_accuracy: 0.9641 - val_loss: 0.1085
Epoch 40/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9402 - loss: 0.1893
- val_accuracy: 0.9647 - val_loss: 0.1099
```

```
Epoch 41/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9375 - loss: 0.1860
- val_accuracy: 0.9659 - val_loss: 0.1106
Epoch 42/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9409 - loss: 0.1853
- val_accuracy: 0.9678 - val_loss: 0.0994
Epoch 43/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9405 - loss: 0.1786
- val_accuracy: 0.9666 - val_loss: 0.1081
Epoch 44/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9419 - loss: 0.1740
- val_accuracy: 0.9656 - val_loss: 0.1055
Epoch 45/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9428 - loss: 0.1824
- val_accuracy: 0.9681 - val_loss: 0.1058
Epoch 46/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9458 - loss: 0.1636
- val_accuracy: 0.9672 - val_loss: 0.1001
Epoch 47/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9426 - loss: 0.1730
- val_accuracy: 0.9653 - val_loss: 0.0984
Epoch 48/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9424 - loss: 0.1742
- val_accuracy: 0.9638 - val_loss: 0.1073
Epoch 49/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9408 - loss: 0.1798
- val_accuracy: 0.9659 - val_loss: 0.0964
Epoch 50/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9487 - loss: 0.1655
- val_accuracy: 0.9678 - val_loss: 0.0939
Epoch 51/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9458 - loss: 0.1605
- val_accuracy: 0.9669 - val_loss: 0.0916
Epoch 52/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9483 - loss: 0.1573
- val_accuracy: 0.9691 - val_loss: 0.0958
Epoch 53/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9477 - loss: 0.1589
- val_accuracy: 0.9647 - val_loss: 0.1019
Epoch 54/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9471 - loss: 0.1708
- val_accuracy: 0.9684 - val_loss: 0.0939
Epoch 55/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9471 - loss: 0.1617
- val_accuracy: 0.9634 - val_loss: 0.1028
Epoch 56/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9455 - loss: 0.1626
- val_accuracy: 0.9688 - val_loss: 0.0952
Epoch 57/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9477 - loss: 0.1606
- val_accuracy: 0.9663 - val_loss: 0.0996
Epoch 58/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9491 - loss: 0.1635
- val_accuracy: 0.9675 - val_loss: 0.1003
Epoch 59/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9529 - loss: 0.1429
- val_accuracy: 0.9725 - val_loss: 0.0874
Epoch 60/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9488 - loss: 0.1576
- val_accuracy: 0.9672 - val_loss: 0.0923
```

```
Epoch 61/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9477 - loss: 0.1626
- val_accuracy: 0.9663 - val_loss: 0.0980
Epoch 62/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9487 - loss: 0.1605
- val_accuracy: 0.9700 - val_loss: 0.0934
Epoch 63/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9507 - loss: 0.1573
- val_accuracy: 0.9700 - val_loss: 0.0911
Epoch 64/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9478 - loss: 0.1612
- val_accuracy: 0.9725 - val_loss: 0.0840
Epoch 65/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9516 - loss: 0.1499
- val_accuracy: 0.9697 - val_loss: 0.0861
Epoch 66/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9526 - loss: 0.1476
- val_accuracy: 0.9712 - val_loss: 0.0885
Epoch 67/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9487 - loss: 0.1510
- val_accuracy: 0.9694 - val_loss: 0.0940
Epoch 68/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9525 - loss: 0.1532
- val_accuracy: 0.9716 - val_loss: 0.0902
Epoch 69/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9553 - loss: 0.1435
- val_accuracy: 0.9722 - val_loss: 0.0870
Epoch 70/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9540 - loss: 0.1496
- val_accuracy: 0.9688 - val_loss: 0.0976
Epoch 71/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9541 - loss: 0.1456
- val_accuracy: 0.9697 - val_loss: 0.0898
Epoch 72/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9526 - loss: 0.1445
- val_accuracy: 0.9697 - val_loss: 0.0985
Epoch 73/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9536 - loss: 0.1419
- val_accuracy: 0.9712 - val_loss: 0.0912
Epoch 74/100
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.9502 - loss: 0.1548
- val_accuracy: 0.9703 - val_loss: 0.0937
```

Training Vs Validation Loss

In [25]:
```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title("Trainig Vs Validation Loss")
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.grid(True)

plt.show()
```

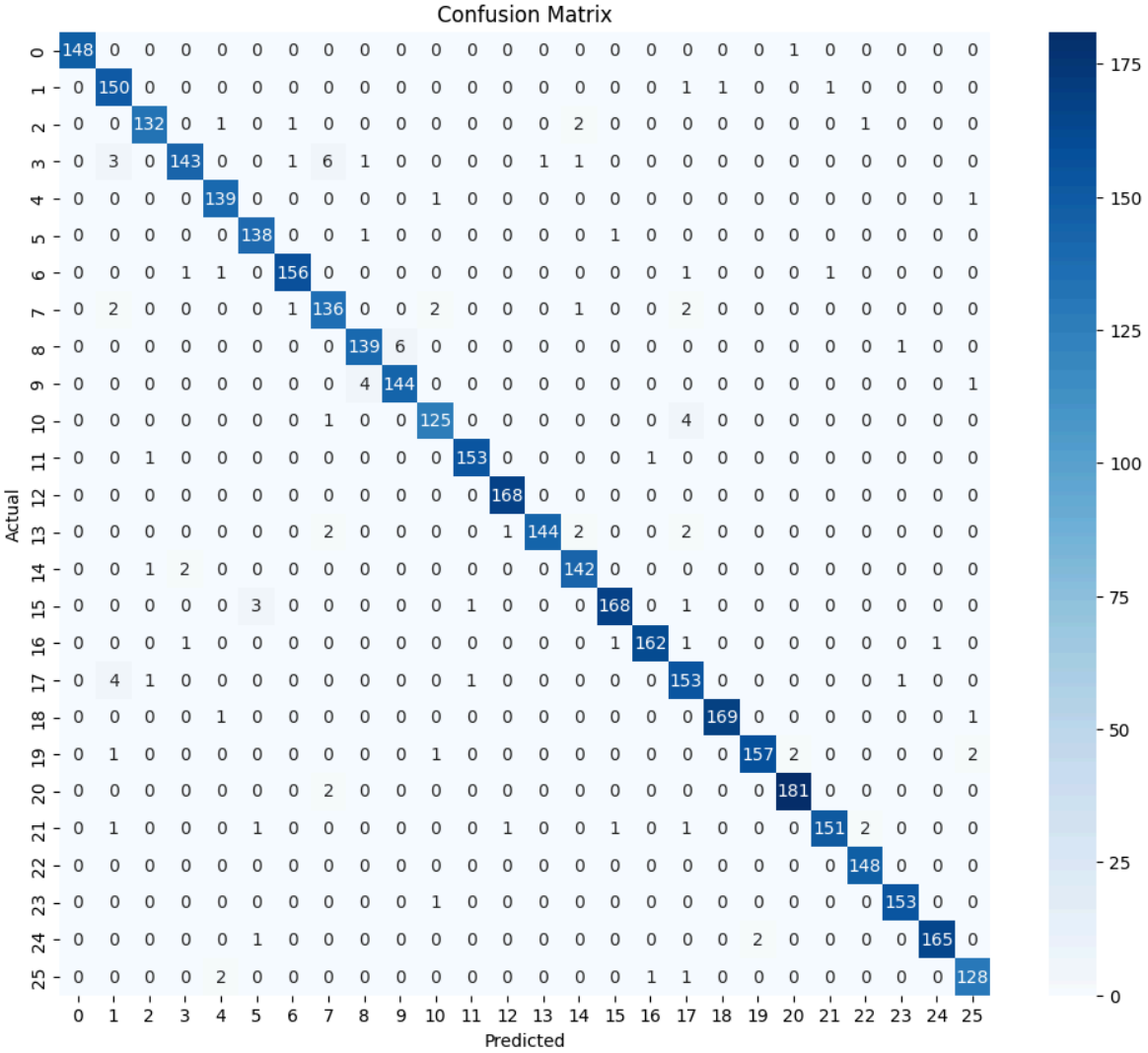


Prediction and Evaluation

In [31]:
```
# Predict probabilities for each class
# Example: [[0.1, 0.7, 0.2], [0.8, 0.1, 0.1]]
y_pred = model.predict(X_test)

# Convert probabilities → predicted class indices
# Example: [0.1, 0.7, 0.2] → 1  (index of max value)
# Result: [1, 0]
y_pred_classes = np.argmax(y_pred, axis=1)

# Convert one-hot actual labels → class indices
# Example: [0,1,0] → 1 , [1,0,0] → 0
# Result: [1, 0]
y_true = np.argmax(y_test, axis=1)

# Compare predicted vs actual → compute accuracy
# Example: y_true = [1,0], y_pred = [1,0] → Accuracy = 2/2 = 1.0
print('Accuracy Score: ', accuracy_score(y_true, y_pred_classes))
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 354us/step
Accuracy Score:  0.973
```

Confusion Matrix

In [41]:
```
cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(12, 10))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title('Confusion Matrix')
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')

plt.show()
```